PRAKTIKUM MONGODB
BASIS DATA NON RELATIONAL

Dosen : Isbat Uzzin Nadhori S.Kom., M.T.



Disusun Oleh:

Faishal Izzuddin Robbani - 3323600017

POLITEKNIK ELEKTRONIKA NEGERI SURABAYA

SAINS DATA TERAPAN

Tujuan

Setelah menyelesaikan praktikum ini, mahasiswa diharapkan mampu:

1. Memahami hubungan MongoDB Server, database, collection, document, dan field.
2. Menginstal atau menyiapkan lingkungan MongoDB.
3. Mengakses MongoDB melalui mongosh dan MongoDB Compass.
4. Membuat database kampus dan collection mahasiswa.
5. Memasukkan, membaca, memperbarui, dan menghapus document (CRUD).
6. Menggunakan query, operator perbandingan, operator logika, query array, dan projection.
7. Mengurutkan hasil, membatasi hasil, dan menerapkan pagination sederhana.
8. Membuat index dan memeriksa rencana eksekusi query.
9. Menggunakan aggregation pipeline untuk menghasilkan ringkasan akademik.
10. Mendokumentasikan hasil praktikum dan mengevaluasi rancangan document.

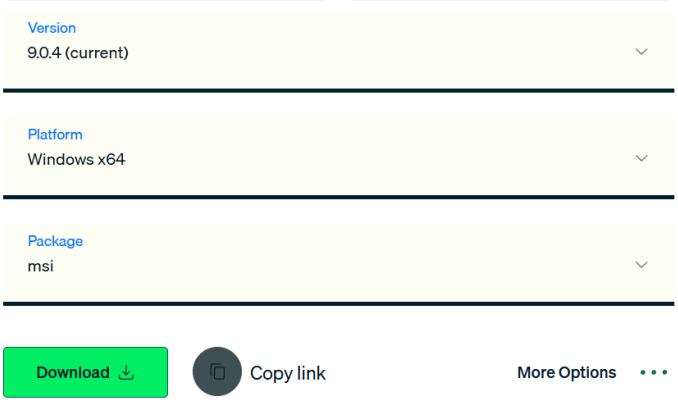
Lingkungan Praktikum

Praktikum ini dilakukan menggunakan sistem operasi Windows 11 dengan MongoDB Community Server versi 9.0.4, MongoDB Shell (mongosh) versi 2.13.0, dan MongoDB Compass versi 1.53.0. Visual Studio Code (VS Code) digunakan sebagai editor kode sekaligus terminal untuk menjalankan perintah MongoDB melalui mongosh. Koneksi dilakukan ke server MongoDB lokal menggunakan alamat mongodb://127.0.0.1:27017. Database yang digunakan adalah kampus dengan collection mahasiswa. MongoDB Shell digunakan untuk menjalankan operasi CRUD, query, indexing, dan aggregation, sedangkan MongoDB Compass digunakan untuk mengelola serta memvisualisasikan data melalui antarmuka grafis.

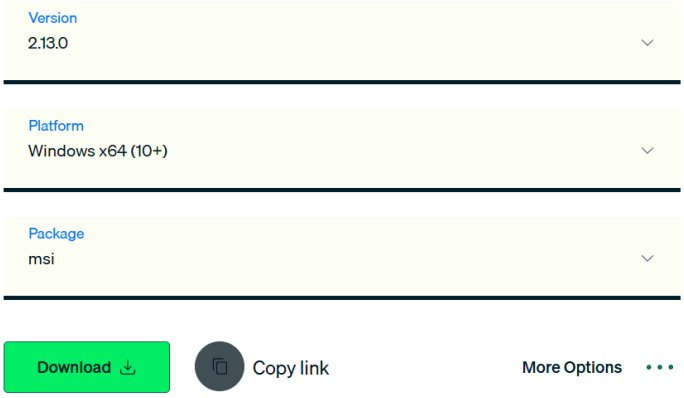
Langkah Instalasi dan Verifikasi

Instalasi dilakukan pada sistem operasi Windows 11 dengan memasang MongoDB Community Server versi 9.0.4 dari web, MongoDB Shell (mongosh) versi 2.13.0 dari web, dan MongoDB Compass versi 1.53.0 yang otomatis terinstall setelah install MongoDB. MongoDB Community Server dikonfigurasi sebagai Windows Service agar dapat berjalan secara lokal, sedangkan mongosh digunakan melalui terminal Visual Studio Code untuk menjalankan perintah basis data. Verifikasi koneksi dilakukan dengan menjalankan perintah mongosh, yang berhasil terhubung ke server lokal pada alamat mongodb://127.0.0.1:27017. Selanjutnya, perintah db.runCommand({ ping: 1 }) menghasilkan nilai ok: 1, yang menunjukkan bahwa server MongoDB dapat diakses dan merespons perintah dengan baik.

Download MongoDB:



Download Mongosh:



Verifikasi Koneksi:

```
PS E:\Kuliah\semester 7\non relational database\praktikum_mongodb> mongosh
Current Mongosh Log ID: 6ad862d8bebe448a4f766bdb
Connecting to:          mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.13.0
Using MongoDB:          9.0.4
Using Mongosh:          2.13.0

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

------
   The server generated these startup warnings when booting
   2026-10-21T13:15:56.739+07:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
------

test> db.runCommand({ ping: 1})
{ ok: 1 }
```

Rancangan Document Mahasiswa

Pada praktikum ini, MongoDB digunakan untuk menyimpan data mahasiswa dalam bentuk dokumen pada collection mahasiswa di database kampus. Setiap dokumen merepresentasikan satu mahasiswa dengan atribut berupa NIM, nama, program studi, tahun angkatan, IPK, kota asal, dan keterampilan yang dikuasai. Struktur dokumen menggunakan format BSON yang mendukung berbagai tipe data, seperti string, angka, dan array.

Rancangan dokumen mahasiswa yang digunakan adalah sebagai berikut:

```
{
 "nim": "3323600024",
 "nama": "Munaffidzur",
 "prodi": "Sains Data",
 "angkatan": 2023,
 "ipk": 3.65,
 "kota": "Sidoarjo",
 "skills": ["Python", "SQL", "MongoDB"]
}
```

Field nim digunakan sebagai identitas unik mahasiswa, nama menyimpan nama mahasiswa, prodi menunjukkan program studi, angkatan menyimpan tahun masuk, ipk menyimpan indeks prestasi kumulatif, dan kota menunjukkan kota asal. Sementara itu, skills berupa array yang digunakan untuk menyimpan beberapa keterampilan dalam satu dokumen.

Struktur tersebut bersifat fleksibel sehingga dokumen dapat ditambahkan atribut lain, misalnya email, sesuai kebutuhan praktikum tanpa harus menetapkan struktur tabel yang kaku seperti pada basis data relasional.

Implementasi Database dan Collection

Dalam MongoDB, database merupakan tempat untuk mengelompokkan berbagai collection. Collection adalah kumpulan dokumen yang memiliki tujuan penyimpanan data serupa, seperti collection mahasiswa untuk menyimpan data mahasiswa. Document merupakan satu kesatuan data yang disimpan dalam format BSON, dengan struktur yang menyerupai objek JSON. Sementara itu, field adalah atribut atau pasangan nama dan nilai di dalam sebuah dokumen, misalnya nim, nama, prodi, dan ipk.

```
test> use kampus
switched to db kampus
kampus> db
kampus
kampus> db.createCollection("mahasiswa")
{ ok: 1 }
kampus> show collections
mahasiswa
```

Hasil CRUD

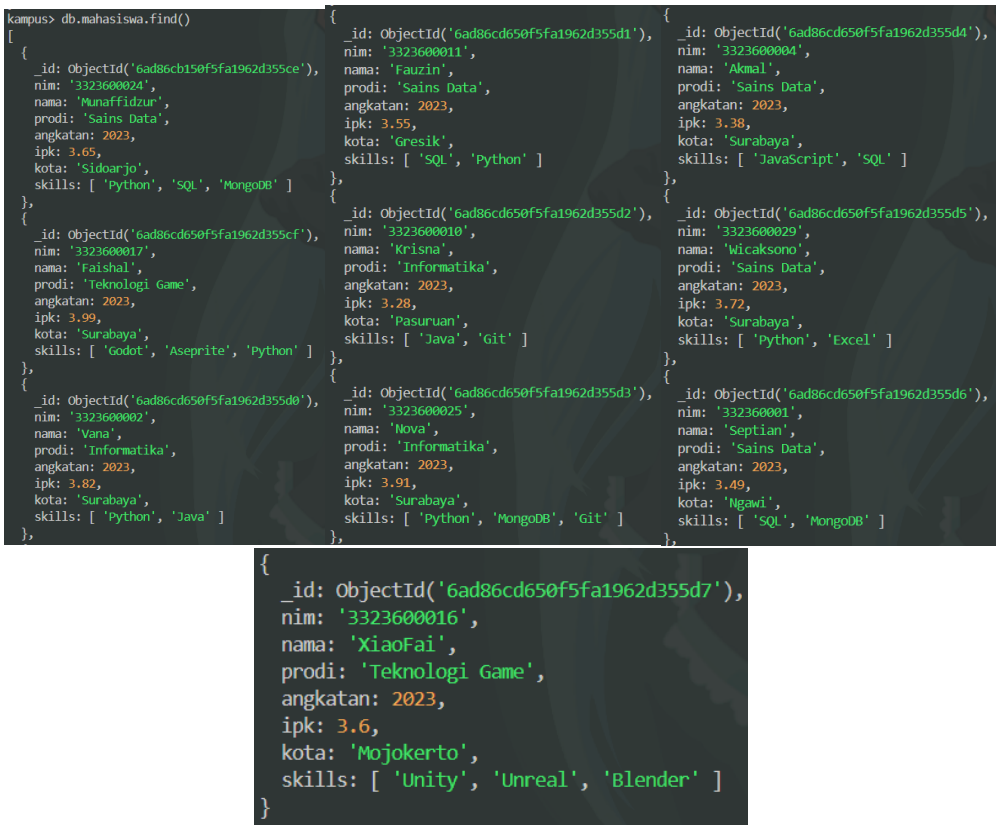
1. Masukkan minimal 10 document mahasiswa.

```
kampus> db.mahasiswa.insertOne({
|   nim: "3323600024",
|   nama: "Munaffidzur",
|   prodi: "Sains Data",
|   angkatan: 2023,
|   ipk: 3.65,
|   kota: "Sidoarjo",
|   skills: ["Python", "SQL", "MongoDB"]
| })
{
  acknowledged: true,
  insertedId: ObjectId('6ad86cb150f5fa1962d355ce')
}
```

```
kampus> db.mahasiswa.insertMany([
|   {
|     nim: "3323600017",
|     nama: "Faishal",
|     prodi: "Teknologi Game",
|     angkatan: 2023,
|     ipk: 3.99,
|     kota: "Surabaya",
|     skills: ["Godot", "Aseprite", "Python"]
|   },
```

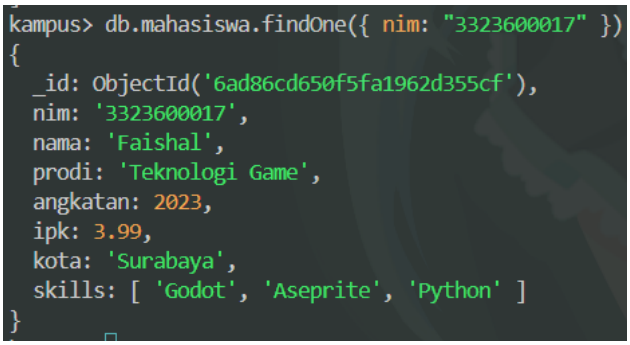
```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6ad86cd650f5fa1962d355cf'),
    '1': ObjectId('6ad86cd650f5fa1962d355d0'),
    '2': ObjectId('6ad86cd650f5fa1962d355d1'),
    '3': ObjectId('6ad86cd650f5fa1962d355d2'),
    '4': ObjectId('6ad86cd650f5fa1962d355d3'),
    '5': ObjectId('6ad86cd650f5fa1962d355d4'),
    '6': ObjectId('6ad86cd650f5fa1962d355d5'),
    '7': ObjectId('6ad86cd650f5fa1962d355d6'),
    '8': ObjectId('6ad86cd650f5fa1962d355d7')
  }
}
```

2. Tampilkan semua mahasiswa.



```
kampus> db.mahasiswa.find()
[
  {
    _id: ObjectId('6ad86cb150f5fa1962d355ce'),
    nim: '3323600024',
    nama: 'Munaffidzur',
    prodi: 'Sains Data',
    angkatan: 2023,
    ipk: 3.65,
    kota: 'Sidoarjo',
    skills: [ 'Python', 'SQL', 'MongoDB' ]
  },
  {
    _id: ObjectId('6ad86cd650f5fa1962d355cf'),
    nim: '3323600017',
    nama: 'Faishal',
    prodi: 'Teknologi Game',
    angkatan: 2023,
    ipk: 3.99,
    kota: 'Surabaya',
    skills: [ 'Godot', 'Aseprite', 'Python' ]
  },
  {
    _id: ObjectId('6ad86cd650f5fa1962d355d0'),
    nim: '3323600002',
    nama: 'Vana',
    prodi: 'Informatika',
    angkatan: 2023,
    ipk: 3.82,
    kota: 'Surabaya',
    skills: [ 'Python', 'Java' ]
  },
  {
    _id: ObjectId('6ad86cd650f5fa1962d355d1'),
    nim: '3323600011',
    nama: 'Fauzin',
    prodi: 'Sains Data',
    angkatan: 2023,
    ipk: 3.55,
    kota: 'Gresik',
    skills: [ 'SQL', 'Python' ]
  },
  {
    _id: ObjectId('6ad86cd650f5fa1962d355d2'),
    nim: '3323600010',
    nama: 'Krisna',
    prodi: 'Informatika',
    angkatan: 2023,
    ipk: 3.28,
    kota: 'Pasuruan',
    skills: [ 'Java', 'Git' ]
  },
  {
    _id: ObjectId('6ad86cd650f5fa1962d355d3'),
    nim: '3323600025',
    nama: 'Nova',
    prodi: 'Informatika',
    angkatan: 2023,
    ipk: 3.91,
    kota: 'Surabaya',
    skills: [ 'Python', 'MongoDB', 'Git' ]
  },
  {
    _id: ObjectId('6ad86cd650f5fa1962d355d4'),
    nim: '3323600004',
    nama: 'Akmal',
    prodi: 'Sains Data',
    angkatan: 2023,
    ipk: 3.38,
    kota: 'Surabaya',
    skills: [ 'JavaScript', 'SQL' ]
  },
  {
    _id: ObjectId('6ad86cd650f5fa1962d355d5'),
    nim: '3323600029',
    nama: 'Wicaksono',
    prodi: 'Sains Data',
    angkatan: 2023,
    ipk: 3.72,
    kota: 'Surabaya',
    skills: [ 'Python', 'Excel' ]
  },
  {
    _id: ObjectId('6ad86cd650f5fa1962d355d6'),
    nim: '332360001',
    nama: 'Septian',
    prodi: 'Sains Data',
    angkatan: 2023,
    ipk: 3.49,
    kota: 'Ngawi',
    skills: [ 'SQL', 'MongoDB' ]
  },
  {
    _id: ObjectId('6ad86cd650f5fa1962d355d7'),
    nim: '3323600016',
    nama: 'XiaoFai',
    prodi: 'Teknologi Game',
    angkatan: 2023,
    ipk: 3.6,
    kota: 'Mojokerto',
    skills: [ 'Unity', 'Unreal', 'Blender' ]
  }
```

3. Cari satu mahasiswa berdasarkan NIM.



```
kampus> db.mahasiswa.findOne({ nim: "3323600017" })
{
  _id: ObjectId('6ad86cd650f5fa1962d355cf'),
  nim: '3323600017',
  nama: 'Faishal',
  prodi: 'Teknologi Game',
  angkatan: 2023,
  ipk: 3.99,
  kota: 'Surabaya',
  skills: [ 'Godot', 'Aseprite', 'Python' ]
}
```

4. Ubah IPK seorang mahasiswa.



```
{
  _id: ObjectId('6ad86cd650f5fa1962d355d1'),
  nim: '3323600011',
  nama: 'Fauzin',
  prodi: 'Sains Data',
  angkatan: 2023,
  ipk: 3.55,
  kota: 'Gresik',
  skills: [ 'SQL', 'Python' ]
},
```

```
kampus> db.mahasiswa.updateOne(
|   { nim: "3323600011" },
|   { $set: { ipk: 3.80 } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

```
kampus> db.mahasiswa.findOne({ nim: "3323600011" })
{
  _id: ObjectId('6ad86cd650f5fa1962d355d1'),
  nim: '3323600011',
  nama: 'Fauzin',
  prodi: 'Sains Data',
  angkatan: 2023,
  ipk: 3.8,
  kota: 'Gresik',
  skills: [ 'SQL', 'Python' ]
}
```

5. Tambahkan field email pada salah satu document.



```
{
  _id: ObjectId('6ad86cd650f5fa1962d355d7'),
  nim: '3323600016',
  nama: 'XiaoFai',
  prodi: 'Teknologi Game',
  angkatan: 2023,
  ipk: 3.6,
  kota: 'Mojokerto',
  skills: [ 'Unity', 'Unreal', 'Blender' ]
}
```

```
kampus> db.mahasiswa.updateOne(
|   { nim: "3323600016" },
|   { $set: { email: "xiaofai@example.com" } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

```
kampus> db.mahasiswa.findOne({ nim: "3323600016" })
{
  _id: ObjectId('6ad86cd650f5fa1962d355d7'),
  nim: '3323600016',
  nama: 'XiaoFai',
  prodi: 'Teknologi Game',
  angkatan: 2023,
  ipk: 3.6,
  kota: 'Mojokerto',
  skills: [ 'Unity', 'Unreal', 'Blender' ],
  email: 'xiaofai@example.com'
}
```

6. Tambahkan skill baru ke array skills.

```
kampus> db.mahasiswa.findOne({ nim: "3323600016" })
{
  _id: ObjectId('6ad86cd650f5fa1962d355d7'),
  nim: '3323600016',
  nama: 'XiaoFai',
  prodi: 'Teknologi Game',
  angkatan: 2023,
  ipk: 3.6,
  kota: 'Mojokerto',
  skills: [ 'Unity', 'Unreal', 'Blender' ],
  email: 'xiaofai@example.com'
}
```

```
kampus> db.mahasiswa.updateOne(
|   { nim: "3323600016" },
|   { $push: { skills: "Aseprite" } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

```
kampus> db.mahasiswa.findOne({ nim: "3323600016" })
{
  _id: ObjectId('6ad86cd650f5fa1962d355d7'),
  nim: '3323600016',
  nama: 'XiaoFai',
  prodi: 'Teknologi Game',
  angkatan: 2023,
  ipk: 3.6,
  kota: 'Mojokerto',
  skills: [ 'Unity', 'Unreal', 'Blender', 'Aseprite' ],
  email: 'xiaofai@example.com'
}
```

7. Hapus satu document berdasarkan NIM.

```
kampus> db.mahasiswa.countDocuments()
10
kampus> db.mahasiswa.deleteOne({ nim: "332360001" })
{ acknowledged: true, deletedCount: 1 }
kampus> db.mahasiswa.findOne({ nim: "332360001" })
null
kampus> db.mahasiswa.countDocuments()
9
```

Hasil query, projection dan sorting

1. Cari mahasiswa dengan IPK minimal 3,50.

```
kampus> db.mahasiswa.find({ ipk: { $gte: 3.50 } })
[
  {
    _id: ObjectId('6ad86cb150f5fa1962d355ce'),
    nim: '3323600024',
    nama: 'Munaffidzur',
    prodi: 'Sains Data',
    angkatan: 2023,
    ipk: 3.65,
    kota: 'Sidoarjo',
    skills: [ 'Python', 'SQL', 'MongoDB' ]
  },
  {
    _id: ObjectId('6ad86cd650f5fa1962d355cf'),
    nim: '3323600017',
    nama: 'Faishal',
    prodi: 'Teknologi Game',
    angkatan: 2023,
    ipk: 3.99,
    kota: 'Surabaya',
    skills: [ 'Godot', 'Aseprite', 'Python' ]
  },
  {
    _id: ObjectId('6ad86cd650f5fa1962d355d0'),
    nim: '3323600002',
    nama: 'Vana',
    prodi: 'Informatika',
    angkatan: 2023,
    ipk: 3.82,
    kota: 'Surabaya',
    skills: [ 'Python', 'Java' ]
  },
  {
    _id: ObjectId('6ad86cd650f5fa1962d355d1'),
    nim: '3323600011',
    nama: 'Fauzin',
    prodi: 'Sains Data',
    angkatan: 2023,
    ipk: 3.8,
    kota: 'Gresik',
    skills: [ 'SQL', 'Python' ]
  },
  {
    _id: ObjectId('6ad86cd650f5fa1962d355d3'),
    nim: '3323600025',
    nama: 'Nova',
    prodi: 'Informatika',
    angkatan: 2023,
    ipk: 3.91,
    kota: 'Surabaya',
    skills: [ 'Python', 'MongoDB', 'Git' ]
  },
  {
    _id: ObjectId('6ad86cd650f5fa1962d355d5'),
    nim: '3323600029',
    nama: 'Wicaksono',
    prodi: 'Sains Data',
    angkatan: 2023,
    ipk: 3.72,
    kota: 'Surabaya',
    skills: [ 'Python', 'Excel' ]
  },
  {
    _id: ObjectId('6ad86cd650f5fa1962d355d7'),
    nim: '3323600016',
    nama: 'XiaoFai',
    prodi: 'Teknologi Game',
    angkatan: 2023,
    ipk: 3.6,
    kota: 'Mojokerto',
    skills: [ 'Unity', 'Unreal', 'Blender', 'Aseprite' ],
    email: 'xiaofai@example.com'
  }
]
```

2. Cari mahasiswa dari program studi Informatika.

```
kampus> db.mahasiswa.find({ prodi: "Informatika" })
[
  {
    _id: ObjectId('6ad86cd650f5fa1962d355d0'),
    nim: '3323600002',
    nama: 'Vana',
    prodi: 'Informatika',
    angkatan: 2023,
    ipk: 3.82,
    kota: 'Surabaya',
    skills: [ 'Python', 'Java' ]
  },
  {
    _id: ObjectId('6ad86cd650f5fa1962d355d2'),
    nim: '3323600010',
    nama: 'Krisna',
    prodi: 'Informatika',
    angkatan: 2023,
    ipk: 3.28,
    kota: 'Pasuruan',
    skills: [ 'Java', 'Git' ]
  },
  {
    _id: ObjectId('6ad86cd650f5fa1962d355d3'),
    nim: '3323600025',
    nama: 'Nova',
    prodi: 'Informatika',
    angkatan: 2023,
    ipk: 3.91,
    kota: 'Surabaya',
    skills: [ 'Python', 'MongoDB', 'Git' ]
  }
]
```

3. Cari mahasiswa angkatan 2023.

```
kampus> db.mahasiswa.find({ angkatan: 2023 })
[
  {
    _id: ObjectId('6ad86cb150f5fa1962d355ce'),
    nim: '3323600024',
    nama: 'Munaffidzur',
    prodi: 'Sains Data',
    angkatan: 2023,
    ipk: 3.65,
    kota: 'Sidoarjo',
    skills: [ 'Python', 'SQL', 'MongoDB' ]
  },
  {
    _id: ObjectId('6ad86cd650f5fa1962d355cf'),
    nim: '3323600017',
    nama: 'Faishal',
    prodi: 'Teknologi Game',
    angkatan: 2023,
    ipk: 3.99,
    kota: 'Surabaya',
    skills: [ 'Godot', 'Aseprite', 'Python' ]
  },
  {
    _id: ObjectId('6ad86cd650f5fa1962d355d0'),
    nim: '3323600002',
    nama: 'Vana',
    prodi: 'Informatika',
    angkatan: 2023,
    ipk: 3.82,
    kota: 'Surabaya',
    skills: [ 'Python', 'Java' ]
  },
```

4. Cari mahasiswa dengan IPK antara 3,00 dan 3,60.

```
kampus> db.mahasiswa.find({
|   ipk: {
|     $gte: 3.00,
|     $lte: 3.60
|   }
| })
[
  {
    _id: ObjectId('6ad86cd650f5fa1962d355d2'),
    nim: '3323600010',
    nama: 'Krisna',
    prodi: 'Informatika',
    angkatan: 2023,
    ipk: 3.28,
    kota: 'Pasuruan',
    skills: [ 'Java', 'Git' ]
  },
  {
    _id: ObjectId('6ad86cd650f5fa1962d355d4'),
    nim: '3323600004',
    nama: 'Akmal',
    prodi: 'Sains Data',
    angkatan: 2023,
    ipk: 3.38,
    kota: 'Surabaya',
    skills: [ 'JavaScript', 'SQL' ]
  },
  {
    _id: ObjectId('6ad86cd650f5fa1962d355d7'),
    nim: '3323600016',
    nama: 'XiaoFal',
    prodi: 'Teknologi Game',
    angkatan: 2023,
    ipk: 3.6,
    kota: 'Mojokerto',
    skills: [ 'Unity', 'Unreal', 'Blender', 'Aseprite' ],
    email: 'xiaofal@example.com'
  }
]
```

5. Cari mahasiswa dari Surabaya.

```
kampus> db.mahasiswa.find({ kota: "Surabaya" })
[
  {
    _id: ObjectId('6ad86cd650f5fa1962d355cf'),
    nim: '3323600017',
    nama: 'Faishal',
    prodi: 'Teknologi Game',
    angkatan: 2023,
    ipk: 3.99,
    kota: 'Surabaya',
    skills: [ 'Godot', 'Aseprite', 'Python' ]
  },
  {
    _id: ObjectId('6ad86cd650f5fa1962d355d0'),
    nim: '3323600002',
    nama: 'Vana',
    prodi: 'Informatika',
    angkatan: 2023,
    ipk: 3.82,
    kota: 'Surabaya',
    skills: [ 'Python', 'Java' ]
  },
  {
    _id: ObjectId('6ad86cd650f5fa1962d355d3'),
    nim: '3323600025',
    nama: 'Nova',
    prodi: 'Informatika',
    angkatan: 2023,
    ipk: 3.91,
    kota: 'Surabaya',
    skills: [ 'Python', 'MongoDB', 'Git' ]
  },
```

6. Cari mahasiswa yang memiliki skill Python.

```
kampus> db.mahasiswa.find({ skills: "Python" })
[
  {
    _id: ObjectId('6ad86cb150f5fa1962d355ce'),
    nim: '3323600024',
    nama: 'Munaffidzur',
    prodi: 'Sains Data',
    angkatan: 2023,
    ipk: 3.65,
    kota: 'Sidoarjo',
    skills: [ 'Python', 'SQL', 'MongoDB' ]
  },
  {
    _id: ObjectId('6ad86cd650f5fa1962d355cf'),
    nim: '3323600017',
    nama: 'Faishal',
    prodi: 'Teknologi Game',
    angkatan: 2023,
    ipk: 3.99,
    kota: 'Surabaya',
    skills: [ 'Godot', 'Aseprite', 'Python' ]
  },
  {
    _id: ObjectId('6ad86cd650f5fa1962d355d0'),
    nim: '3323600002',
    nama: 'Vana',
    prodi: 'Informatika',
    angkatan: 2023,
    ipk: 3.82,
    kota: 'Surabaya',
    skills: [ 'Python', 'Java' ]
  },
  {
    _id: ObjectId('6ad86cd650f5fa1962d355d1'),
    nim: '3323600011',
    nama: 'Fauzin',
    prodi: 'Sains Data',
    angkatan: 2023,
    ipk: 3.8,
    kota: 'Gresik',
    skills: [ 'SQL', 'Python' ]
  },
  {
    _id: ObjectId('6ad86cd650f5fa1962d355d3'),
    nim: '3323600025',
    nama: 'Nova',
    prodi: 'Informatika',
    angkatan: 2023,
    ipk: 3.91,
    kota: 'Surabaya',
    skills: [ 'Python', 'MongoDB', 'Git' ]
  },
  {
    _id: ObjectId('6ad86cd650f5fa1962d355d5'),
    nim: '3323600029',
    nama: 'Wicaksono',
    prodi: 'Sains Data',
    angkatan: 2023,
    ipk: 3.72,
    kota: 'Surabaya',
    skills: [ 'Python', 'Excel' ]
  }
]
```

7. Cari mahasiswa Informatika dengan IPK minimal 3,50.

```
kampus> db.mahasiswa.find({
|   prodi: "Informatika",
|   ipk: { $gte: 3.50 }
| })
[
  {
    _id: ObjectId('6ad86cd650f5fa1962d355d0'),
    nim: '3323600002',
    nama: 'Vana',
    prodi: 'Informatika',
    angkatan: 2023,
    ipk: 3.82,
    kota: 'Surabaya',
    skills: [ 'Python', 'Java' ]
  },
  {
    _id: ObjectId('6ad86cd650f5fa1962d355d3'),
    nim: '3323600025',
    nama: 'Nova',
    prodi: 'Informatika',
    angkatan: 2023,
    ipk: 3.91,
    kota: 'Surabaya',
    skills: [ 'Python', 'MongoDB', 'Git' ]
  }
]
```

8. Tampilkan hanya nama, prodi, dan ipk.

```
kampus> db.mahasiswa.find(
|   {},
|   { _id: 0, nama: 1, prodi: 1, ipk: 1 }
| )
[
  { nama: 'Munaffidzur', prodi: 'Sains Data', ipk: 3.65 },
  { nama: 'Faishal', prodi: 'Teknologi Game', ipk: 3.99 },
  { nama: 'Vana', prodi: 'Informatika', ipk: 3.82 },
  { nama: 'Fauzin', prodi: 'Sains Data', ipk: 3.8 },
  { nama: 'Krisna', prodi: 'Informatika', ipk: 3.28 },
  { nama: 'Nova', prodi: 'Informatika', ipk: 3.91 },
  { nama: 'Akmal', prodi: 'Sains Data', ipk: 3.38 },
  { nama: 'Wicaksono', prodi: 'Sains Data', ipk: 3.72 },
  { nama: 'XiaoFai', prodi: 'Teknologi Game', ipk: 3.6 }
]
```

9. Urutkan mahasiswa berdasarkan IPK tertinggi.

```
kampus> db.mahasiswa.find().sort({ ipk: -1 })
[
  {
    _id: ObjectId('6ad86cd650f5fa1962d355cf'),
    nim: '3323600017',
    nama: 'Faishal',
    prodi: 'Teknologi Game',
    angkatan: 2023,
    ipk: 3.99,
    kota: 'Surabaya',
    skills: [ 'Godot', 'Aseprite', 'Python' ]
  },
  {
    _id: ObjectId('6ad86cd650f5fa1962d355d3'),
    nim: '3323600025',
    nama: 'Nova',
    prodi: 'Informatika',
    angkatan: 2023,
    ipk: 3.91,
    kota: 'Surabaya',
    skills: [ 'Python', 'MongoDB', 'Git' ]
  },
  {
    _id: ObjectId('6ad86cd650f5fa1962d355d0'),
    nim: '3323600002',
    nama: 'Vana',
    prodi: 'Informatika',
    angkatan: 2023,
    ipk: 3.82,
    kota: 'Surabaya',
    skills: [ 'Python', 'Java' ]
  },
  {
    _id: ObjectId('6ad86cd650f5fa1962d355d1'),
    nim: '3323600011',
    nama: 'Fauzin',
    prodi: 'Sains Data',
    angkatan: 2023,
    ipk: 3.8,
    kota: 'Gresik',
    skills: [ 'SQL', 'Python' ]
  },
  {
    _id: ObjectId('6ad86cd650f5fa1962d355d5'),
    nim: '3323600029',
    nama: 'Wicaksono',
    prodi: 'Sains Data',
    angkatan: 2023,
    ipk: 3.72,
    kota: 'Surabaya',
    skills: [ 'Python', 'Excel' ]
  },
  {
    _id: ObjectId('6ad86cb150f5fa1962d355ce'),
    nim: '3323600024',
    nama: 'Munaffidzur',
    prodi: 'Sains Data',
    angkatan: 2023,
    ipk: 3.65,
    kota: 'Sidoarjo',
    skills: [ 'Python', 'SQL', 'MongoDB' ]
  },
  {
    _id: ObjectId('6ad86cd650f5fa1962d355d7'),
    nim: '3323600016',
    nama: 'XiaoFai',
    prodi: 'Teknologi Game',
    angkatan: 2023,
    ipk: 3.6,
    kota: 'Mojokerto',
    skills: [ 'Unity', 'Unreal', 'Blender', 'Aseprite' ],
    email: 'xiaofai@example.com'
  },
  {
    _id: ObjectId('6ad86cd650f5fa1962d355d4'),
    nim: '3323600004',
    nama: 'Akmal',
    prodi: 'Sains Data',
    angkatan: 2023,
    ipk: 3.38,
    kota: 'Surabaya',
    skills: [ 'JavaScript', 'SQL' ]
  },
  {
    _id: ObjectId('6ad86cd650f5fa1962d355d2'),
    nim: '3323600010',
    nama: 'Krisna',
    prodi: 'Informatika',
    angkatan: 2023,
    ipk: 3.28,
    kota: 'Pasuruan',
    skills: [ 'Java', 'Git' ]
  }
```

10. Tampilkan lima mahasiswa dengan IPK tertinggi.

```
kampus> db.mahasiswa.find().sort({ ipk: -1 }).limit(5)
[
  {
    _id: ObjectId('6ad86cd650f5fa1962d355cf'),
    nim: '3323600017',
    nama: 'Faishal',
    prodi: 'Teknologi Game',
    angkatan: 2023,
    ipk: 3.99,
    kota: 'Surabaya',
    skills: [ 'Godot', 'Aseprite', 'Python' ]
  },
  {
    _id: ObjectId('6ad86cd650f5fa1962d355d3'),
    nim: '3323600025',
    nama: 'Nova',
    prodi: 'Informatika',
    angkatan: 2023,
    ipk: 3.91,
    kota: 'Surabaya',
    skills: [ 'Python', 'MongoDB', 'Git' ]
  },
  {
    _id: ObjectId('6ad86cd650f5fa1962d355d0'),
    nim: '3323600002',
    nama: 'Vana',
    prodi: 'Informatika',
    angkatan: 2023,
    ipk: 3.82,
    kota: 'Surabaya',
    skills: [ 'Python', 'Java' ]
  },
  {
    _id: ObjectId('6ad86cd650f5fa1962d355d1'),
    nim: '3323600011',
    nama: 'Fauzin',
    prodi: 'Sains Data',
    angkatan: 2023,
    ipk: 3.8,
    kota: 'Gresik',
    skills: [ 'SQL', 'Python' ]
  },
  {
    _id: ObjectId('6ad86cd650f5fa1962d355d5'),
    nim: '3323600029',
    nama: 'Wicaksono',
    prodi: 'Sains Data',
    angkatan: 2023,
    ipk: 3.72,
    kota: 'Surabaya',
    skills: [ 'Python', 'Excel' ]
  }
```

Hasil indexing dan explain( )

1. Tampilkan index yang tersedia.

```
kampus> db.mahasiswa.getIndexes()
[ { v: 2, key: { _id: 1 }, name: '_id_' } ]
```

2. Buat unique index pada nim.

```
kampus> db.mahasiswa.createIndex(
|   { nim: 1 },
|   { unique: true }
| )
nim_1
kampus> db.mahasiswa.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { nim: 1 }, name: 'nim_1', unique: true }
]
```

3. Buat compound index pada prodi dan ipk.

```
kampus> db.mahasiswa.createIndex({
|   prodi: 1,
|   ipk: -1
| })
prodi_1_ipk_-1
kampus> db.mahasiswa.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { nim: 1 }, name: 'nim_1', unique: true },
  { v: 2, key: { prodi: 1, ipk: -1 }, name: 'prodi_1_ipk_-1' }
]
```

4. Jalankan explain("executionStats") untuk query berdasarkan prodi.

```
kampus> db.mahasiswa.find({
|   prodi: "Informatika"
| }).explain("executionStats")
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'kampus.mahasiswa',
    parsedQuery: { prodi: { '$eq': 'Informatika' } },
    indexFilterSet: false,
    queryHash: 'F77E794D',
    planCacheShapeHash: 'F77E794D',
    planCacheKey: 'ACA33F90',
    optimizationTimeMillis: 8,
    optimizationTimeMicros: 8190,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'FETCH',
      nss: 'kampus.mahasiswa',
      inputStage: {
        stage: 'IXSCAN',
        nss: 'kampus.mahasiswa',
        keyPattern: { prodi: 1, ipk: -1 },
        indexName: 'prodi_1_ipk_-1',
        isMultiKey: false,
        multiKeyPaths: { prodi: [], ipk: [] },
        isUnique: false,
        isSparse: false,
        isPartial: false,
        indexVersion: 2,
        direction: 'forward',
        indexBounds: {
          prodi: [ '["Informatika", "Informatika"]' ],
          ipk: [ '[MaxKey, MinKey]' ]
        }
      }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 3,
    executionTimeMillis: 15,
    executionTimeMicros: 15026,
    totalKeysExamined: 3,
    totalDocsExamined: 3,
    executionStages: {
      isCached: false,
      stage: 'FETCH',
      nReturned: 3,
      executionTimeMillisEstimate: 0,
      works: 4,
      advanced: 3,
      needTime: 0,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
      nss: 'kampus.mahasiswa',
      docsExamined: 3,
      alreadyHasObj: 0,
      inputStage: {
        stage: 'IXSCAN',
        nReturned: 3,
        executionTimeMillisEstimate: 0,
        works: 4,
        advanced: 3,
        needTime: 0,
        needYield: 0,
        saveState: 0,
        restoreState: 0,
        isEOF: 1,
        nss: 'kampus.mahasiswa',
        keyPattern: { prodi: 1, ipk: -1 },
        indexName: 'prodi_1_ipk_-1',
        isMultiKey: false,
        multiKeyPaths: { prodi: [], ipk: [] },
        isUnique: false,
        isSparse: false,
```



Hasil aggregation

1. Hitung jumlah mahasiswa per prodi.



2. Hitung rata-rata IPK per prodi.



3. Cari IPK tertinggi dan terendah per prodi.

```
kampus> db.mahasiswa.aggregate([
|   {
|     $group: {
|       _id: "$prodi",
|       ipk_tertinggi: { $max: "$ipk" },
|       ipk_terendah: { $min: "$ipk" }
|     }
|   },
|   {
|     $sort: { _id: 1 }
|   }
| ])
[
  { _id: 'Informatika', ipk_tertinggi: 3.91, ipk_terendah: 3.28 },
  { _id: 'Sains Data', ipk_tertinggi: 3.8, ipk_terendah: 3.38 },
  { _id: 'Teknologi Game', ipk_tertinggi: 3.99, ipk_terendah: 3.6 }
]
```

4. Buat pipeline yang menggabungkan $match, $group, dan $sort.

```
kampus> db.mahasiswa.aggregate([
|   {
|     $match: {
|       ipk: { $gte: 3.50 }
|     }
|   },
|   {
|     $group: {
|       _id: "$prodi",
|       jumlah_mahasiswa: { $sum: 1 },
|       rata_rata_ipk: { $avg: "$ipk" }
|     }
|   },
|   {
|     $sort: {
|       jumlah_mahasiswa: -1,
|       _id: 1
|     }
|   }
| ])
[
  {
    _id: 'Sains Data',
    jumlah_mahasiswa: 3,
    rata_rata_ipk: 3.723333333333333
  },
  { _id: 'Informatika', jumlah_mahasiswa: 2, rata_rata_ipk: 3.865 },
  { _id: 'Teknologi Game', jumlah_mahasiswa: 2, rata_rata_ipk: 3.795 }
]
```

Jawaban Analisis

1. Mengapa MongoDB disebut document database?
   MongoDB disebut document database karena data disimpan dalam bentuk dokumen BSON yang menyerupai objek JSON. Setiap dokumen berisi pasangan field dan value, misalnya data mahasiswa yang memiliki field nim, nama, prodi, dan ipk. Dokumen-dokumen tersebut disimpan di dalam collection.

2. Apa perbedaan JSON dan BSON?
   JSON (JavaScript Object Notation) merupakan format pertukaran data berbasis teks yang mudah dibaca manusia. Sementara itu, BSON (Binary JSON) merupakan format biner yang digunakan MongoDB untuk menyimpan dokumen. BSON mendukung tipe data tambahan, seperti tanggal dan ObjectId, serta menyediakan representasi data yang sesuai untuk kebutuhan penyimpanan dan pemrosesan MongoDB.

3. Mengapa NIM cocok dibuat sebagai unique index?
   NIM cocok dijadikan unique index karena setiap mahasiswa seharusnya memiliki NIM yang berbeda. Dengan opsi unique: true, MongoDB akan menolak penyimpanan NIM duplikat sehingga identitas mahasiswa lebih konsisten dan dapat dicari secara efisien.

4. Apa manfaat dan risiko schema flexibility?
   Schema flexibility memungkinkan dokumen dalam satu collection memiliki field yang berbeda sesuai kebutuhan tanpa harus mengubah struktur seluruh collection. Contohnya, field email dapat ditambahkan pada dokumen mahasiswa tertentu tanpa harus menambahkannya pada semua dokumen. Namun, fleksibilitas ini juga berisiko menimbulkan ketidakkonsistenan nama field, tipe data, atau kelengkapan informasi jika tidak disertai aturan validasi yang baik.

5. Kapan embedding cocok digunakan? Berikan contoh dari document mahasiswa.
   Embedding cocok digunakan ketika data tambahan memiliki hubungan erat dengan dokumen utama dan biasanya dibaca atau diperbarui bersama. Contohnya, field skills pada dokumen mahasiswa menyimpan beberapa keterampilan dalam bentuk array, seperti ["Python", "SQL", "MongoDB"]. Keterampilan tersebut dapat disimpan langsung dalam dokumen mahasiswa karena merupakan informasi yang berkaitan dengan mahasiswa tersebut.

6. Mengapa index tidak sebaiknya dibuat pada semua field?
   Index dapat mempercepat pencarian, tetapi setiap index membutuhkan ruang penyimpanan dan perlu diperbarui ketika data ditambahkan, diubah, atau dihapus. Jika index dibuat pada semua field, penggunaan penyimpanan dan biaya operasi penulisan dapat meningkat. Oleh karena itu, index sebaiknya dibuat pada field yang sering digunakan untuk pencarian, pengurutan, atau kebutuhan query tertentu.

7. Apa informasi yang dapat dibaca dari explain("executionStats")?
   explain("executionStats") memberikan informasi mengenai cara MongoDB menjalankan query. Informasi yang dapat diamati antara lain jenis tahapan eksekusi, apakah query menggunakan index atau melakukan pemindaian collection, jumlah dokumen yang dikembalikan (nReturned), jumlah entri index yang diperiksa (totalKeysExamined), jumlah dokumen yang diperiksa (totalDocsExamined), dan waktu eksekusi yang dilaporkan (executionTimeMillis). Informasi tersebut dapat digunakan untuk mengevaluasi efisiensi query.

8. Apa manfaat aggregation untuk laporan akademik?
   Aggregation bermanfaat untuk mengolah data mahasiswa menjadi ringkasan yang lebih mudah dianalisis. Misalnya, MongoDB dapat menghitung jumlah mahasiswa per program studi, rata-rata IPK, serta IPK tertinggi dan terendah pada setiap program studi. Hasilnya dapat digunakan untuk menyusun laporan statistik akademik tanpa harus menghitung data secara manual satu per satu.

Kesimpulan

Berdasarkan praktikum yang telah dilakukan, MongoDB merupakan basis data NoSQL yang menyimpan data dalam bentuk dokumen sehingga struktur datanya lebih fleksibel. Melalui praktikum ini, saya telah mempelajari pembuatan database dan collection, pengelolaan data menggunakan operasi CRUD, serta penerapan query, projection, sorting, indexing, dan aggregation. Saya juga memahami bahwa penggunaan index dapat membantu proses pencarian data, sedangkan aggregation berguna untuk mengolah data menjadi informasi yang lebih ringkas. Dengan demikian, praktikum ini membantu saya memahami dasar penggunaan MongoDB dalam mengelola dan menganalisis data mahasiswa.